

OS LAB 9 TASKS

Syed Sultan Ahmed, 24K-0585

BCS-4H

1)

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>

void sigint_handler(int signum) {
    fprintf(stdout, " Caught Signal %d", signum);
}

int main() {
    signal(SIGINT, sigint_handler);
    printf("Press Ctrl C to terminate\n");

    while(1) {
        sleep(1);
    }

    return 0;
}
```

```
sultan@sultan-VirtualBox:~/Downloads/OS/Lab09$ gcc 1_basic-SIGINT.c -o output
sultan@sultan-VirtualBox:~/Downloads/OS/Lab09$ ./output
Press Ctrl C to terminate
^C^C
```

```
sultan@sultan-VirtualBox:~$ pgrep output
16336
sultan@sultan-VirtualBox:~$ kill -9 16336
sultan@sultan-VirtualBox:~$
```

2)

```
#define _XOPEN_SOURCE 700
#include <stdio.h>
```

```

#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void sighandler(int signum) {
    printf("SigInt Recieved: %d", signum);
    exit(signum);
}

int main() {
    struct sigaction sa;
    sa.sa_handler = sighandler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGINT, &sa, NULL);
    while(1) {}
    return EXIT_SUCCESS;
}

```

Killed

```

sultan@sultan-VirtualBox:~/Downloads/OS/Lab09$ gcc 2_sigaction.c -o output && ./output
^CSigInt Recieved: 2sultan@sultan-VirtualBox:~/Downloads/OS/Lab09$ 

```

3)

```

#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <sys/wait.h>
#include <sys/resource.h>

void proc_wait() {
    int wstat;
    pid_t pid;
    while ((pid = waitpid(-1, &wstat, WNOHANG)) > 0) {
        printf("Return Code: %d\n", wstat);
    }
}

```

```

int main() {
    signal(SIGCHLD, proc_wait);
    for (int i = 0; i < 3; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            printf("Child %d started\n", getpid());
            sleep(2);
            printf("Child %d exiting\n", getpid());
            exit(0);
        }
    }
    sleep(10);
    printf("Parent exiting\n");
    return 0;
}

```

```

Child 16852 started
Child 16853 started
Child 16854 started
Child 16853 exiting
Child 16852 exiting
Return Code: 0
Parent exiting
Child 16854 exiting

```

4)

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <signal.h>
#include <unistd.h>

#define NUM_THREADS 4
pthread_t threads[NUM_THREADS]; // Global variable to store thread IDs

void sigusr1_handler(int signum) { // signal handler
    int i = gettid(), j = getppid(), k = getpid();
}

```

```

    fprintf(stdout, "Thread %u received SIGUSR1 signal (parent=%u) [pid=%u]
{tid=%u} \n", pthread_self(), j, k, i);
}

void *thread_function(void *arg) { // thread function
    signal(SIGUSR1, sigusr1_handler); // register signal handler
    while (1) sleep(1);                // Keep the thread alive
    return NULL;
}

int main() {
    signal(SIGUSR1, sigusr1_handler); // register signal handler
    for (int i = 0; i < NUM_THREADS; ++i) {
        if (pthread_create(&threads[i], NULL, thread_function, NULL) != 0)
        {
            perror("pthread_create"); exit(EXIT_FAILURE);
        }
    }

    int i = gettid(), j = getpid(), k = getppid();
    fprintf(stdout, "Parent Process ID: %u, Process ID:%u, main() thread
ID: %u\n", k, j, i);
    fprintf(stdout, "Thread IDs: 0=%u, 1=%u, 2=%u and 3=%lu\n", threads[0],
threads[1], threads[2], threads[3]);

    printf("Sending Signal to Process\n");
    kill(j, SIGUSR1); // send signal to process
    printf("Sending Signal to Thread 3\n");
    pthread_kill(threads[2], SIGUSR1); // send signal to 3rd thread

    for (int i = 0; i < NUM_THREADS; ++i) {
        if (pthread_join(threads[i], NULL) != 0) {
            perror("pthread_join"); exit(EXIT_FAILURE);
        }
    }
    return 0;
}

```

```
Thread IDs: 0=4269799104, 1=4261406400, 2=4253013696 and 3=127947025401536
Sending Signal to Process
Thread 4272707392 received SIGUSR1 signal (parent=16276) [pid=16953] {tid=16953}
Sending Signal to Thread 3
Thread 4253013696 received SIGUSR1 signal (parent=16276) [pid=16953] {tid=16956}
```

5)

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <signal.h>
#include <unistd.h>
// key insight: it doesn't matter which thread registers the handler, it
// applies process-wide. The runtime will find whichever thread has the
// handler registered and use it.
pthread_t threads[3];
void handler(int sig) {
    printf("Signal %d handled by thread %lu\n", sig, pthread_self());
}
void* worker(void* arg) {
    int id = *(int*)arg;
    signal(SIGUSR1, handler); // uncommented as req by c)
    printf("Thread %d started (TID: %lu)\n", id, pthread_self());
    while (1) {
        sleep(1);
    }
    return NULL;
}
int main() {
    int ids[3] = {1, 2, 3};
    // signal(SIGUSR1, handler); // commented as req by c)
    for (int i = 0; i < 3; i++) {
        pthread_create(&threads[i], NULL, worker, &ids[i]);
    }
    sleep(2);
    printf("\nSending signal to PROCESS\n");
    kill(getpid(), SIGUSR1);
    sleep(2);
    printf("\nSending signal to THREAD 3\n");
```

```
pthread_kill(threads[2], SIGUSR1);  
sleep(2);  
return 0;  
}
```

Thread 1 started (TID: 130158533015232)

Thread 2 started (TID: 130158524622528)

Thread 3 started (TID: 130158516229824)

Sending signal to PROCESS

Signal 10 handled by thread 130158536193856

Sending signal to THREAD 3

Signal 10 handled by thread 130158516229824